

Guide complet de soutenance — DR-DOCSCOL

Projet : Application de demande et retrait de documents scolaires (OBC / DECC)

Public : Lecteur débutant (niveau très inférieur) pouvant présenter le projet à la place du porteur

Date : Juin 2026

Version : 1.0

Comment utiliser ce document

1. Lire d'abord les sections 1 à 4 (vue d'ensemble).
2. Étudier la section 5 (backend) puis la section 6 (frontend).
3. Mémoriser les règles métier (section 7).
4. S'entraîner avec les questions du jury (section 8).
5. Utiliser le glossaire (section 9) quand un mot technique apparaît.

Fichier Word associé :

docs/documents-word/GUIDE_SOUTENANCE_COMPLET_NIVEAU_DEBUTANT.docx (même contenu, format Word).

1. Qu'est-ce que DR-DOCSCOL ? (explication simple)

Imagine une **mairie numérique pour les diplômes scolaires** au Cameroun :

- Un **élève** qui a réussi un examen (BEPC, Probatoire, Baccalauréat) veut récupérer son **diplôme**, son **relevé de notes** ou un **duplicata** (copie en cas de perte).
- Deux organismes gèrent ces documents : **OBC** (Office du Baccalauréat du Cameroun) et **DECC** (pour le BEPC).
- L'application **DR-DOCSCOL** permet de :
 - se connecter en ligne ;
 - voir l'état de ses documents ;
 - faire une demande ;
 - payer un duplicata (simulation) ;
 - prendre un **rendez-vous** pour retirer le document au **centre d'examen** ou à l'**antenne régionale** ;
- recevoir des **notifications** (email + application).

Trois types d'utilisateurs :

Utilisateur	Rôle technique	Ce qu'il fait
Élève	ELEVE	Consulte, demande, paie, prend RDV
Administrateur OBC ou DECC	ADMINISTRATEUR	Valide, change les statuts, importe des élèves
Agent centre d'examen	AGENT_CENTRE_EXAMEN	Voit les RDV de sa région et confirme le retrait physique

2. Méthodologie du projet (comment le travail a été organisé)

2.1 Approche générale

Le projet suit une démarche **proche du cycle en V simplifié**, adaptée à un projet académique :

Phase	Contenu	Livrables dans le repo
Analyse des besoins	Cahier des charges, acteurs, règles OBC/DECC	docs/ cahier_des_charges_mis_a_jour.md
Conception	Diagrammes UML (cas d'utilisation, classes, séquences, MCD/MLD)	docs/ diagrammes_uml_mis_a_jour.md, docs/diagrammes-images/
Implémentation	Code Next.js + Prisma + Supabase	app/, lib/, prisma/
Tests	Tests manuels, scripts seed, guides curl	docs/ GUIDE_TEST_FONCTIONNEL.md, docs/CURL_TEST_EXAMPLES.md
Documentation	Guides API, état du projet, soutenance	docs/

2.2 Techniques de conception utilisées

- **UML (Unified Modeling Language)** : langage de dessin pour expliquer le système avant de coder.
- **Diagramme de cas d'utilisation** : qui fait quoi (élève, admin, agent).
- **Diagramme de classes** : entités (User, Document, RendezVous...).
- **Diagrammes de séquence** : ordre des appels (ex. : connexion → base → redirection).
- **MCD/MLD** : modèle conceptuel et logique de la base de données.
- **Architecture en couches** :
 1. **Présentation** : pages React (ce que l'utilisateur voit).
 2. **Application** : routes API + Server Actions (logique métier).
 3. **Données** : Prisma + PostgreSQL (stockage).
- **Séparation des responsabilités** : chaque dossier a un rôle clair (lib/ = logique réutilisable, app/api/ = API HTTP, components/ = interface).

2.3 Méthodes de développement

- **Itérations fonctionnelles** : d'abord auth, puis documents, puis RDV, puis paiements, puis agent centre.
- **Données de test (seed)** : scripts pour créer admins, élèves, agents sans tout saisir à la main.
- **Validation des entrées** : bibliothèque **Zod** pour refuser les données incorrectes avant traitement.
- **Traçabilité** : table **AuditLog** pour enregistrer les actions sensibles (changement de statut, retrait confirmé).

2.4 Choix méthodologiques à dire au jury

« Nous avons d'abord modélisé le métier avec UML, puis implémenté une architecture web moderne type **full-stack JavaScript** avec séparation claire entre interface, API et

base. Les règles OBC/DECC sont centralisées dans un module de routage pour éviter les incohérences. »

3. Langages et technologies (liste complète)

3.1 Langages

Langage	Où ?	Pourquoi ?
TypeScript	Partout dans le code source	JavaScript avec typage : moins d'erreurs, meilleure autocomplétion
SQL	Migrations Prisma	Créer/modifier les tables PostgreSQL
HTML/CSS	Rendu des pages	Structure et style (via Tailwind)
Markdown	Documentation docs/	Rédaction des guides

3.2 Frameworks et bibliothèques principales

Technologie	Rôle en une phrase
Next.js 15	Framework web : pages, API, rendu serveur
React 19	Bibliothèque d'interface utilisateur (composants)
Prisma 6	ORM : parler à PostgreSQL avec des objets TypeScript
PostgreSQL	Base de données relationnelle (hébergée chez Supabase)
Supabase Auth	Gestion des mots de passe et sessions
Tailwind CSS	Classes utilitaires pour le design
Radix UI / shadcn	Composants accessibles (boutons, dialogues, selects)
Zod	Validation des formulaires et JSON API
Nodemailer	Envoi d'emails (SMTP / Gmail)
Framer Motion	Animations sur la page d'accueil

3.3 Outils DevOps / qualité

- **Node.js 20+** : exécution du serveur
- **npm** : gestion des paquets
- **ESLint / Prettier** : qualité et formatage du code
- **tsx** : exécution des scripts TypeScript (seed)
- **Vercel / Docker** (prévu) : déploiement

4. Architecture globale du système

[Navigateur élève / admin / agent]

```

      v
[ Next.js - Frontend + Backend dans le même projet ]
|
|    +--> Server Components (pages qui lisent la BDD)
|    +--> Client Components ('use client', formulaires)
|    +--> Server Actions ('use server', formulaires sécurisés)
|    +--> Route Handlers (app/api/.../route.ts = API REST)
|
+--> Middleware (rafraîchit session Supabase)
|
v
[ Supabase Auth ]   [ PostgreSQL via Prisma ]
(login/logout)      (users, documents, RDV...)

```

Point important pour le jury : ce n'est pas deux applications séparées (front Angular + back Spring). C'est une **application Next.js full-stack** : le « backend » vit dans le même projet que le « frontend ».

5. BACKEND — Tout expliquer simplement

Le « backend » ici = tout ce qui ne s'affiche pas directement à l'écran : base de données, règles métier, API, emails, sécurité.

5.1 La base de données (Prisma + PostgreSQL)

Fichier principal : `prisma/schema.prisma`

Principe : On décrit les tables en langage Prisma ; Prisma génère le client TypeScript et les migrations SQL.

Tables / modèles importants :

Modèle	Rôle métier
User	Tous les comptes (élève, admin, agent)
ExamenValide	Examen réussi par un élève (BEPC, Probatoire, Bac)
DocumentAcademique	Diplôme, relevé ou ligne duplicata liée à un examen
Duplicata	Dossier de demande de duplicata + paiement
RendezVous	Créneau réservé pour retirer un document
Paielement / RecuPaielement	Paielement duplicata et reçu
Notification	Message in-app pour l'élève
AuditLog	Journal des actions admin/agent
Organisme / AntenneRegionale / CentreExamen	Structure OBC, DECC, régions
ParametreRendezVous	Quota journalier, créneaux horaires

Enums (listes fermées) : `Role`, `StatutDocument`, `StatutRendezVous`, `TypeDocument`, `DiplomePrincipal`, etc.

Relations : Un élève a plusieurs examens ; chaque examen génère des documents ; un document peut avoir des rendez-vous.

Migrations : Dossier `prisma/migrations/` — chaque fichier SQL = une évolution du schéma (ex. ajout notifications, agent centre).

Commandes utiles à citer :

```
`bash
npx prisma migrate deploy # Appliquer les migrations en production
npx prisma studio         # Voir les données visuellement
npm run db:generate       # Régénérer le client Prisma
`
```

5.2 Connexion à la base : lib/prisma.ts

- Crée **une seule instance** Prisma (pattern singleton) pour ne pas ouvrir trop de connexions.
 - Utilise `DATABASE_URL` (pooler) et `DIRECT_URL` (connexion directe) depuis `.env`.
-

5.3 Authentification : Supabase + lib/auth.ts

Pourquoi deux systèmes ?

- **Supabase Auth** : stocke le mot de passe (haché), gère login/logout, cookies de session.
- **Table `User` Prisma** : stocke le métier (matricule, rôle, organisme, antenne, centre).

Flux de connexion :

1. L'utilisateur envoie email + mot de passe.
2. Supabase vérifie et pose un cookie sb-....
3. `getCurrentUser()` lit le cookie, récupère `authUserId`, cherche la ligne dans `User`.
4. Selon le rôle, redirection vers `/dashboard`, `/admin` ou `/centre-examen`.

Fonctions clés :

- `getCurrentUser()` : utilisateur connecté ou null
- `requireUser()` / `requireRole()` : sinon redirection login
- `getHomePathForRole()` : page d'accueil selon le rôle

Fichiers Supabase :

- `lib/supabase/server.ts` : côté serveur (pages, API)
- `lib/supabase/client.ts` : côté navigateur si besoin
- `lib/supabase/middleware.ts` : rafraîchit la session à chaque requête

Middleware racine : `middleware.ts` — bloque l'accès à `/dashboard`, `/admin`, `/account` sans cookie Supabase.

5.4 Routage métier OBC/DECC : lib/document-routing.ts

Cœur métier du backend. Centralise les règles :

- BEPC → organisme **DECC**
- Probatoire / Baccalauréat → organisme **OBC**
- Probatoire : **pas** de diplôme original
- Bac **original** → retrait **antenne régionale OBC**
- BEPC **duplicata** → antenne **DECC**
- Relevés, BEPC original, Probatoire relevé, Bac relevé → **centre d'examen** + rendez-vous

Fonctions importantes :

- `resolveDocumentRoute()` : calcule organisme, antenne, lieu, type de retrait
- `getAntenneForRegion()` : antenne selon la région de composition
- `getCentreExamenForRegion()` : nom du centre régional

- `isCentreExamenPickupDocument()` : le document se retire-t-il au centre ?
- `getAdminDocumentScope()` : filtre les documents visibles par un admin régional

Pourquoi c'est une bonne pratique ? Une seule source de vérité : si la règle change, on modifie un fichier, pas 50 pages.

5.5 Rendez-vous : `lib/appointment-service.ts`

- Créneaux horaires actifs (`ParametreCreneau`)
 - Quota journalier (`ParametreRendezVous`)
 - `getAvailableSlots(date)` : places restantes par créneau
 - `getPickupLocation(document)` : texte du lieu de retrait
 - Jours fériés, week-ends : dates interdites ou limitées
 - Statuts actifs : PLANIFIE, CONFIRME
-

5.6 Agent centre d'examen : `lib/centre-examen-service.ts`

- `getAgentCentreExamen()` : vérifie que l'agent a un `centreExamenId`
 - `getCentreDocumentWhere()` : quels documents concernent ce centre (région + type)
 - `getCentreExamenAppointmentWhere()` : filtres today / upcoming / processed
 - `syncDocumentPickupFromExam()` : aligne région du document sur l'examen avant RDV
 - `isCentreExamenDocumentEligible()` : contrôle avant confirmation de retrait
-

5.7 Duplicatas : `lib/duplicata-service.ts`

- Frais : 10 000 FCFA (relevé), 15 000 FCFA (original)
 - `parseDuplicataInstruction()` : lit le JSON des pièces/motif dans le champ instruction
 - `resolvePickupRouteForDocument()` : route de retrait pour un duplicata validé
 - Délai d'un an entre deux demandes de duplicata (règle métier)
-

5.8 Notifications : `lib/notification-service.ts`

- Crée des lignes dans Notification (titre, message, lien action)
 - Types : disponibilité document, RDV, rappel 30 jours, etc.
-

5.9 Emails : `lib/mail-service.ts` + `lib/mailer.ts` + `lib/email-template.ts`

- **Nodemailer** envoie via SMTP (Gmail configuré dans `.env`)
 - Templates HTML pour : document disponible, RDV confirmé, retrait confirmé, mot de passe oublié
-

5.10 API REST : dossier `app/api/`

Chaque fichier `route.ts` exporte des fonctions HTTP (GET, POST, PATCH...).

Helpers communs : `lib/api-utils.ts`

- `requireApiUser(role?)` : utilisateur connecté + bon rôle
- `parseJson(request, schemaZod)` : corps JSON validé

- `json(data) / handleApiError(error)` : réponses uniformes

Liste des API par domaine :

Chemin	Méthode	Qui ?	Action
/api/auth/login	POST	Public	Connexion
/api/auth/register	POST	Public	Activation élève
/api/auth/logout	POST	Connecté	Déconnexion
/api/auth/password/forgot	POST	Public	Email reset
/api/auth/password/reset	POST	Public	Nouveau mot de passe
/api/students/me/documents	GET	Élève	Liste documents
/api/students/me/documents/[id]	GET	Élève	Détail document
/api/students/me/documents/[id]/appointments	POST	Élève	Créer RDV
/api/students/me/documents/[id]/calendar-event	GET	Élève	Fichier .ics
/api/students/me/payments/initiate	POST	Élève	Lancer paiement
/api/students/me/payments/[id]/cancel	POST	Élève	Annuler paiement
/api/students/me/notifications	GET	Élève	Notifications
/api/admin/documents	GET	Admin	Documents du scope
/api/admin/documents/[id]/status	PATCH	Admin	Changer statut
/api/admin/students	GET	Admin	Élèves
/api/admin/students/import	POST	Admin	Import CSV
/api/admin/withdrawals	GET/POST	Admin	Historique retraits
/api/admin/payments	GET	Admin	Paiements
/api/admin/audit-logs	GET	Admin	Journaux
/api/centre-examen/appointments	GET	Agent	Liste RDV
/api/centre-examen/appointments/[id]/confirm-withdrawal	PATCH	Agent	Confirmer retrait
/api/payments/webhook	POST	Système	Webhook paiement (prévu)
/api/internal/notifications/...	POST	Cron interne	Rappels
/api/health	GET	Public	Santé application

Technique HTTP à connaître :

- **GET** = lire
- **POST** = créer
- **PATCH** = modifier partiellement

- Codes : **200** OK, **201** créé, **400** données invalides, **401** non connecté, **403** interdit, **404** introuvable, **409** conflit métier

5.11 Server Actions : `app/dashboard/actions.ts` et `app/admin/actions.ts`

Mot-clé : 'use server' en haut du fichier.

Différence avec l'API :

- Appelées directement depuis un **formulaire HTML** React (sans écrire fetch manuellement).
- S'exécutent **toujours sur le serveur** : secrets et BDD protégés.

Exemples d'actions élève :

- `reserverDisponibiliteAction` : créer un rendez-vous
- `requestOriginalDiplomaAction` / demande relevé : enregistrer la demande
- `requestDuplicataAction` : dossier duplicata + paiement
- `cancelRendezVousAction` : annuler RDV

Exemples admin :

- Import CSV élèves
- Création manuelle élève
- Mise à jour statuts (selon pages)

Sécurité : Chaque action commence par `getCurrentUser()` et vérifie le rôle.

5.12 Import admin : `lib/admin-student-import.ts`

- Parse un **CSV** avec colonnes normalisées (matricule, email, diplôme, région, statut document, RDV...)
 - Crée ou met à jour User, ExamenValide, DocumentAcademique, RendezVous
 - Respecte le **scope** admin (organisme + antenne)
-

5.13 Scripts utilitaires (scripts/)

Script	Rôle
<code>seed-diplomes.ts</code>	Utilisateur de test Faiza + documents
<code>seed-regional-admins.ts</code>	10 admins OBC
<code>seed-decc-regional-admins.ts</code>	10 admins DECC
<code>seed-centre-examen-agents.ts</code>	10 agents centre
<code>seed-soutenance-eleves.ts</code>	5 élèves demo soutenance
<code>seed-appointments.ts</code>	Paramètres RDV

5.14 Sécurité backend (points jury)

1. **Mots de passe** : jamais en clair dans Prisma ; Supabase les hache.
2. **Contrôle d'accès** : chaque route vérifie rôle + scope (région, organisme).
3. **Validation Zod** : refuse injections et champs manquants.
4. **Transactions Prisma** : réservation RDV en transaction pour éviter double booking.
5. **Audit logs** : traçabilité des changements sensibles.
6. **Limites connues** : paiement simulé, fichiers justificatifs pas encore sur Supabase Storage.

6. FRONTEND — Tout expliquer simplement

Le « frontend » = ce que l'utilisateur voit : pages, boutons, formulaires, tableaux.

6.1 Next.js App Router (structure app/)

Chaque **dossier** = une URL. Fichier `page.tsx` = la page affichée.

Exemples d'URLs :

URL	Fichier	Public
/	app/page.tsx	Landing marketing
/auth/login	app/auth/login/page.tsx	Connexion élève
/auth/login/obc	Admin OBC	
/auth/login/decc	Admin DECC	
/auth/login/centre-examen	Agent centre	
/auth/register	Activation compte	
/dashboard	Espace élève	
/dashboard/documents	Documents	
/dashboard/payments	Paielements	
/dashboard/rendez-vous	RDV	
/admin	Espace admin	
/centre-examen	Espace agent	
/account	Profil	

Fichiers spéciaux :

- app/layout.tsx : mise en page globale (header, thème, toaster)
- loading.tsx / error.tsx : états chargement et erreur (si présents)

6.2 Server Components vs Client Components

Type	Directive	Où ça tourne ?	Exemple
Server Component	(aucune, par défaut)	Serveur	Page liste documents qui lit Prisma
Client Component	'use client'	Navigateur	Formulaire avec état, toast, fetch

Pourquoi cette séparation ?

- Le serveur peut accéder directement à la BDD (plus rapide, plus sécurisé).
- Le client gère l'interactivité (clics, modales, animations).

Règle à dire au jury : « On utilise le serveur par défaut ; le client seulement quand il faut de l'interactivité. »

6.3 Composants réutilisables (components/)

Dossier	Contenu
components/ui/	Boutons, inputs, tables, dialogues (design system)
components/auth/	Formulaires login, register, reset password

components/dashboard/	Sidebar, header, shell des tableaux de bord
components/documents/	Cartes documents, dialogue prise de RDV
components/centre-examen/	Liste RDV agent, bouton confirmer retrait
components/admin/	Formulaires import, gestion élèves
components/landing/	Page d'accueil marketing

`DashboardShell` : enveloppe commune (menu latéral + titre) pour dashboard, admin et centre.

6.4 Styling : Tailwind CSS

- Classes utilitaires dans le JSX : `className="rounded-lg border p-4"`
- Variables CSS pour thème OBC/DECC : `styles/globals.css`
- Responsive : préfixes `sm:`, `lg:` pour mobile/desktop

6.5 Formulaires et validation

1. **HTML form** avec `action={serverAction}` (Server Action)
2. Ou **fetch** vers `/api/...` (Client Component)
3. **Zod** valide côté serveur (jamais faire confiance au navigateur seul)
4. **react-hook-form** + **@hookform/resolvers** sur certains formulaires complexes

Feedback utilisateur :

- `react-toastify` : messages succès/erreur
- `action-loading-dialog` : spinner pendant l'action

6.6 Parcours écran par écran (à démontrer)

Élève :

1. Activation (`/auth/register`) avec matricule pré-enregistré par admin
2. Login → `/dashboard`
3. Documents : voir statuts, demander relevé, duplicata
4. Paiement duplicata → reçu PDF/texte
5. RDV : choisir date (à partir de demain), créneau, confirmer
6. Notifications : lire, supprimer

Admin :

1. Login OBC ou DECC + sélection région si besoin
2. Liste documents → passer en DISPONIBLE → email élève
3. Import CSV ou création manuelle élève
4. Consultation paiements, audit

Agent centre :

1. Login centre
2. Onglet **À venir** : voir RDV des élèves de sa région
3. Jour J : onglet **Aujourd'hui** → **Confirmer retrait** → document RETIRE

6.7 Page d'accueil (landing)

- `components/landing/landing-page.tsx` : présentation marketing, parcours, FAQ

- Animations Framer Motion, ancres de navigation
- Liens vers login élève / OBC / DECC

7. Règles métier à réciter sans hésitation

Document	Organisme	Lieu de retrait	RDV ?
BEPC original	DECC	Centre d'examen	Oui
BEPC relevé	DECC	Centre d'examen	Oui
BEPC duplicata	DECC	Antenne régionale	Non (workflow admin)
Probatoire relevé	OBC	Centre d'examen	Oui
Probatoire original	—	Non délivré	—
Bac relevé	OBC	Centre d'examen	Oui
Bac original	OBC	Antenne régionale OBC	Oui

Statuts document : PAS_DISPONIBLE → DISPONIBLE → RETIRE

Statuts RDV : PLANIFIE → (optionnel CONFIRME) → HONORE ou ANNULE

8. Questions du jury — avec réponses prêtes à lire

8.1 Méthodologie et gestion de projet

Q1 : Quelle méthodologie avez-vous utilisée ?

R : Une approche en cycle en V adaptée : analyse (cahier des charges), conception UML, implémentation itérative par modules (auth, documents, RDV, paiements), tests manuels et jeux de données seed, documentation continue dans docs/.

Q2 : Pourquoi avoir fait des diagrammes UML avant le code ?

R : Pour valider la compréhension du métier avec le encadrant, fixer le vocabulaire (acteurs, entités) et guider le schéma Prisma. Les diagrammes de séquence aident à expliquer les flux à la soutenance.

Q3 : Comment avez-vous géré les exigences changeantes ?

R : Le cahier des charges a été versionné (cahier_des_charges_mis_a_jour.md). Les règles OBC/DECC sont centralisées dans document-routing.ts pour limiter l'impact des changements.

Q4 : Quels livrables produisez-vous ?

R : Code source, migrations BDD, documentation technique, guides de test, diagrammes, scripts seed, guide de soutenance.

8.2 Architecture et techniques

Q5 : Pourquoi Next.js et pas seulement React ?

R : Next.js ajoute le routage par fichiers, le rendu serveur, les API intégrées et l'optimisation production. Un seul projet full-stack simplifie le déploiement.

Q6 : Qu'est-ce qu'une Server Action ?

R : Une fonction marquée 'use server' appelée depuis le frontend mais exécutée sur le serveur, idéale pour les formulaires sécurisés sans exposer la logique métier.

Q7 : Différence entre Route Handler et Server Action ?

R : Route Handler (app/api/.../route.ts) = API REST JSON pour fetch/AJAX ou intégrations externes. Server Action = formulaires et mutations depuis les pages React.

Q8 : Pourquoi Prisma ?

R : Prisma est un ORM typé : le schéma génère des types TypeScript, réduit les erreurs SQL et facilite les migrations versionnées.

Q9 : Pourquoi Supabase si vous avez déjà Prisma ?

R : Supabase Auth gère l'authentification (sessions, reset password) ; PostgreSQL Supabase stocke les données métier accessibles via Prisma. On sépare « qui est connecté » (Supabase) et « profil métier » (table User).

Q10 : Comment protégez-vous les routes ?

R : Triple niveau : middleware (cookie session), requireRole dans les pages serveur, requireApiUser dans les API.

8.3 Base de données

Q11 : Combien de tables principales ?

R : Environ une vingtaine de modèles Prisma (users, examens, documents, duplicatas, rendez-vous, paiements, notifications, organismes, antennes, centres, audit, paramètres...). Voir diagramme classes v2.

Q12 : Qu'est-ce qu'une migration ?

R : Un fichier SQL versionné qui modifie le schéma de façon reproductible (prisma migrate).

Q13 : Comment liez-vous un RDV à un document ?

R : Le champ documentId sur RendezVous avec clé étrangère vers DocumentAcademique.

Q14 : Comment isolez-vous les admins régionaux ?

R : Champs organismeId et antenneRegionaleId sur User et Document ; filtre getAdminDocumentScope().

8.4 Métier et fonctionnel

Q15 : Qui crée les rendez-vous ?

R : L'élève, quand le document est DISPONIBLE et nécessite un RDV. L'agent centre ne crée pas ; il consulte et confirme le retrait physique.

Q16 : Pourquoi l'agent ne voit-il pas le Bac original ?

R : Le Bac original se retire à l'antenne OBC, pas au centre d'examen. Le filtre getCentreDocumentWhere exclut ce cas.

Q17 : Comment fonctionne le duplicata ?

R : L'élève soumet motif + pièces + paiement simulé ; l'admin valide le dossier ; le statut passe à DISPONIBLE ; retrait selon la route (souvent antenne pour BEPC).

Q18 : Le paiement est-il réel ?

R : Non en production complète : flux applicatif avec statut EFFECTUE et génération de reçu ; webhook préparé pour Orange Money / MTN / carte plus tard.

8.5 Qualité, tests, limites

Q19 : Quels tests avez-vous ?

R : Tests manuels documentés (GUIDE_TEST_FONCTIONNEL.md), exemples curl

(CURL_TEST_EXAMPLES.md), scripts seed pour jeux de données. Les tests automatisés E2E sont identifiés comme amélioration future.

Q20 : Principales limites du projet ?

R : Paiement non branché à un opérateur réel ; stockage fichiers justificatifs pas encore sur cloud ; connexion pooler Supabase parfois instable en dev ; pas de traduction/rectification diplôme (hors périmètre actuel).

Q21 : Comment déployer ?

R : Build Next.js (npm run build), migrations (prisma migrate deploy), variables d'environnement sur Vercel ou Docker ; Supabase pour Auth et Postgres.

Q22 : Scalabilité ?

R : Architecture stateless côté Next.js ; pool de connexions Postgres ; possibilité CDN pour assets ; séparation Auth/DB déjà compatible cloud.

8.6 Questions pièges — réponses courtes

Q23 : Où est stocké le mot de passe ? → Supabase Auth, pas dans notre table User.

Q24 : Que se passe-t-il si deux élèves réservent le même créneau ? → Transaction + quota par créneau ; le second reçoit une erreur 409.

Q25 : Comment l'élève sait que son document est prêt ? → Notification in-app + email via notifyDocumentAvailable.

Q26 : Différence PLANIFIE et CONFIRME pour un RDV ? → PLANIFIE = réservé ; CONFIRME = variante de confirmation ; l'agent peut honorer les deux.

Q27 : Pourquoi TypeScript ? → Typage fort, moins de bugs, meilleure maintenance.

Q28 : C'est quoi Zod ? → Bibliothèque de validation de schémas pour JSON et formulaires.

9. Glossaire des mots-clés techniques

Mot	Explication simple
API	Interface pour que des programmes échangent des données (ici HTTP JSON).
App Router	Système de routes de Next.js basé sur les dossiers app/.
Audit log	Journal qui enregistre qui a fait quoi et quand.
Authentification	Vérifier l'identité (login).
Autorisation	Vérifier les droits (rôle, région).
Backend	Partie serveur : logique + base + API.
Client Component	Code React exécuté dans le navigateur.
Cookie de session	Petit fichier stocké par le navigateur pour rester connecté.
CRUD	Create, Read, Update, Delete — opérations de base sur les données.
CSV	Fichier texte tableur pour imports en masse.
Enum	Liste fermée de valeurs (ex. statuts).
Frontend	Partie visible : pages et composants.
Full-stack	Frontend + backend dans le même projet.
Hash	Transformation irréversible du mot de passe pour le stocker sans le lire.
HTTP	Protocole du web (GET, POST, etc.).
JSON	Format texte pour échanger des données

	structurées.
Middleware	Code exécuté avant chaque requête (ici refresh session).
Migration	Script SQL de mise à jour de la base.
ORM	Outil qui mappe tables SQL ↔ objets code (Prisma).
PostgreSQL	Système de base de données relationnelle.
REST	Style d'API basé sur URLs et verbes HTTP.
Scope	Périmètre de visibilité (ex. admin ne voit que sa région).
Seed	Script qui remplit la BDD avec données de test.
Server Action	Fonction serveur appelée depuis un formulaire React.
Server Component	Composant rendu sur le serveur.
Session	Période pendant laquelle l'utilisateur reste connecté.
SMTP	Protocole d'envoi d'emails.
Supabase	Service cloud : Auth + Postgres + Storage.
Transaction	Groupe d'opérations BDD : tout réussit ou tout échoue.
TypeScript	JavaScript avec types.
UML	Langage de diagrammes de conception.
Validation	Vérifier que les données entrantes sont correctes.
Webhook	URL appelée automatiquement par un service externe (ex. paiement).
Zod	Bibliothèque de validation de schémas TypeScript.

10. Script oral de 5 minutes (à apprendre par cœur)

« Bonjour. Notre projet **DR-DOCSCOL** digitalise les demandes et retraits de documents scolaires pour l'**OBC** et la **DECC**.

Nous avons trois acteurs : l'**élève**, l'**administrateur régional** et l'**agent de centre d'examen**. L'élève active son compte, consulte ses documents générés depuis ses examens validés, peut demander un relevé ou un duplicata, payer en ligne de façon simulée, et réserver un créneau de retrait. L'admin change les statuts et notifie l'élève. L'agent confirme le retrait physique le jour du rendez-vous.

Techniquement, c'est une application **Next.js 15 full-stack** en **TypeScript** : le frontend utilise React et Tailwind ; le backend utilise des **Route Handlers**, des **Server Actions**, **Prisma** sur **PostgreSQL**, et **Supabase Auth** pour les mots de passe.

Le cœur métier est le module **`document-routing`** qui applique les règles : par exemple le baccalauréat original part à l'antenne OBC, les relevés au centre d'examen avec rendez-vous.

Nous avons documenté le projet avec UML, guides de test et scripts seed. Les limites annoncées sont le paiement mobile réel et le stockage cloud des pièces justificatives.

Merci, nous sommes prêts pour la démonstration et vos questions. »

11. Démonstration recommandée (ordre)

1. Page d'accueil /
2. Connexion élève → documents → statut DISPONIBLE
3. Prise de RDV → vérifier email / notification
4. Connexion agent même région → onglet **À venir** → voir le RDV
5. Connexion admin → passer un document en DISPONIBLE
6. (Optionnel) Demande duplicata + paiement + reçu

Comptes de test : voir docs/ETAT_FINAL_PROJET.md et docs/agents-centres-examen-test.md.

12. Fichiers à connaître absolument

Fichier	Pourquoi
prisma/schema.prisma	Structure de la base
lib/document-routing.ts	Règles OBC/DECC
lib/auth.ts	Sessions et rôles
lib/centre-examen-service.ts	RDV agent centre
app/dashboard/actions.ts	Actions élève
app/api/centre-examen/appointments/route.ts	API agent
middleware.ts	Protection routes
docs/diagrammes_uml_mis_a_jour.md	Schémas à projeter

13. Si le jury demande « Où est le backend ? »

Réponse recommandée :

« Nous n'avons pas un dossier backend/ séparé. Le backend est distribué dans `lib/` pour la logique métier, `app/api/` pour les API REST, `app/*/actions.ts` pour les Server Actions, et `prisma/` pour la persistance. C'est l'architecture recommandée par Next.js App Router.

»

Fin du guide — Bonne soutenance.